

MODULE 15

PASSWORDS & CREDENTIAL SECURITY

Credential lifecycle, password storage, offline cracking in a safe lab, MFA, reuse, detection & remediation

Authorized-lab only

Semua cracking hands-on menggunakan hash/credential dummy yang dibundel di module ini atau data yang memang lo punya izin tertulis untuk audit. Tidak ada password spraying, credential stuffing, atau brute-force terhadap service publik/real account.

Omarchy / Arch Linux Edition

Project: VaxXinE/red-teaming

Version 1.0 | 05 Oktober 2026

Field	Value
Primary OS	Omarchy / Arch Linux
Prerequisite	Modules 00-14
Core tools	John the Ripper, Hashcat, OpenSSL, Python, sha256sum
Hands-on	Dummy local hashes, synthetic auth logs, local policy/audit scripts
Guided labs	15 labs + capstone
Next module	16 - Linux Privilege Escalation

1. Learning Objectives & Safety Contract

- [] Membedakan credential, authenticator, password, hash, token, private key, recovery code, dan session secret.
- [] Menjelaskan hashing vs encryption vs encoding, serta kenapa password storage membutuhkan password KDF.
- [] Memahami salt, pepper, work factor, memory hardness, dan migration/re-hashing strategy.
- [] Membedakan online guessing, brute force, password spraying, credential stuffing, dan offline cracking.
- [] Menggunakan John dan Hashcat hanya pada hash training lokal yang kita kontrol.
- [] Melakukan small wordlist/rule/mask reasoning tanpa mengejar keyspace tidak realistis.
- [] Mengevaluasi password policy modern, reuse, MFA, recovery, dan credential handling.
- [] Membaca synthetic auth telemetry dari perspektif Blue Team.
- [] Menulis finding dan remediation tanpa membocorkan plaintext credential.

Safety contract

Cracking capability dipakai untuk audit resistance of storage/policy pada data training. Online attack terhadap login service tidak dilakukan di module ini. Jika nanti engagement mengizinkan online auth testing, rate, lockout, monitoring, dan stop condition wajib ditentukan di RoE.

Prerequisite

- Module 05: Python/Bash dan evidence hygiene.
- Module 06: scope, RoE, stop conditions, data handling.
- Module 09: vulnerability assessment dan remediation reasoning.
- Module 14: exploit evidence discipline dan minimum sufficient proof.

2. Credential Threat Model

Credential adalah data atau artifact yang membantu membuktikan identity atau memperoleh access. Password hanyalah satu jenis. Dalam assessment profesional, handling credential sering lebih sensitif daripada teknik yang digunakan untuk menemukannya.

Credential / Authenticator	Contoh	Risiko utama
Password / PIN	Memorized secret	Weak/reused/guessed/phished.
Password hash	Verifier database / shadow-like entry	Offline guessing jika database bocor.
Session secret	Cookie / bearer token	Replay selama masih valid.
API token	PAT / service token	Long-lived access tanpa password.
Private key	SSH/TLS signing key	Possession can directly authenticate.
Recovery code	MFA fallback code	Bypasses primary MFA if stolen.
Passkey / security key	Public-key authenticator	Stronger phishing resistance; recovery still matters.

Credential != plaintext password

Jangan menyamakan semua credential dengan password. Hash tidak "didecrypt"; token tidak selalu perlu di-crack; private key punya threat model berbeda. Pilih control berdasarkan jenis authenticator.

Attack surface

Creation -> Storage -> Verification -> Transmission -> Recovery -> Rotation -> Revocation -> Logging

Satu sistem bisa menyimpan password dengan baik tetapi tetap lemah di recovery flow, session handling, atau long-lived service credential.

3. Password Guessability & Modern Policy

Password policy modern berfokus pada length, compromised/common-password blocklist, user choice, password manager compatibility, dan MFA - bukan memaksa pola seperti 1 huruf besar + 1 angka + 1 simbol yang sering menghasilkan password predictable.

NIST SP 800-63B-4 highlights

Area	Guidance ringkas
Single-factor password	Minimum 15 characters.
Password used with MFA	May be shorter, but minimum 8 characters.
Maximum	Should permit at least 64 characters.
Composition rules	Do not require arbitrary character-class mixtures.
Periodic rotation	Do not force routine changes without evidence of compromise.
Blocklist	Reject commonly used, expected, or compromised values.
Unicode / spaces	Support user-friendly long passphrases where implementation allows.

Policy != strength oracle

Lolos policy tidak berarti password kuat. `this is a long passphrase 2026` bisa lolos length, tetapi predictability tetap bergantung pada how it was chosen and whether it appears in known compromised/common sets.

Lab 3.1 - Local policy checker

```
python labs/policy-check.py 'this is a long passphrase 2026'
python labs/policy-check.py password
python labs/policy-check.py 'Longer-But-With-MFA' --mfa
```

4. Encoding vs Encryption vs Hashing

Mechanism	Reversible?	Key required?	Use case
Encoding	Yes	No	Representation/transport, e.g. Base64.
Encryption	Yes	Yes	Confidentiality for data that must be recovered.
Cryptographic hash	No practical inverse	No	Integrity/fingerprint.
Password KDF	No practical inverse	Salt + cost parameters	Password verifier designed to make guessing expensive.

Common mistake

Base64 bukan encryption. Raw SHA-256 adalah cryptographic hash, tetapi terlalu cepat untuk password storage. Password verifier perlu algorithm yang intentionally expensive.

Lab 4.1 - Format observation

```
python labs/hash-identify.py 'VGhpcyBpcyBiYXNlbnQ='
python labs/hash-identify.py '$6$mod15a$example'
python labs/hash-identify.py 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa'
```

Classifier hanya memberi format hint; context tetap diperlukan. String 64 hex character tidak membuktikan itu SHA-256 password hash.

5. Password Storage: Salt, Pepper & KDF

Salt

Salt adalah random per-password value yang disimpan bersama verifier. Salt membuat password sama menghasilkan verifier berbeda dan menghancurkan keuntungan precomputed rainbow-table style lookup.

Pepper

Pepper adalah secret tambahan yang disimpan terpisah dari password database. Pepper merupakan defense-in-depth; ia tidak menggantikan KDF dan lifecycle pepper harus dipikirkan seperti secret cryptographic lain.

Work factor / memory cost

KDF sengaja dibuat mahal agar satu legitimate verification masih acceptable, tetapi jutaan tebakan offline menjadi mahal. Nilai cost harus dituning pada hardware production, dipantau, dan dinaikkan seiring waktu.

Algorithm	Posisi saat ini
Argon2id	Pilihan utama OWASP untuk aplikasi baru bila tersedia; memory-hard.
scrypt	Alternatif memory-hard bila Argon2id tidak tersedia.
bcrypt	Legacy-compatible; perhatikan 72-byte input limit pada banyak implementation.
PBKDF2-HMAC-SHA256	Relevant untuk FIPS/NIST-approved ecosystem; work factor tinggi.
Raw MD5/SHA-1/SHA-256	Tidak cocok untuk password storage karena terlalu cepat.

OWASP current baseline

OWASP Password Storage Cheat Sheet saat ini merekomendasikan Argon2id dan memberi baseline parameter; PBKDF2-HMAC-SHA256 600,000 iterations direkomendasikan ketika PBKDF2/FIPS diperlukan. Selalu cek versi guidance terbaru saat implementasi production.

6. Lab - Same Password, Different Salt

Lab ini menggunakan PBKDF2-HMAC-SHA256 built-in Python agar tidak membutuhkan dependency tambahan. Tujuannya melihat role salt dan work factor, bukan menetapkan framework production.

```
python labs/pbkdf2-demo.py \  
--password 'Correct Horse Battery Staple'
```

Expected observation: dua verifier berbeda karena salt berbeda, walaupun password sama. Verification tetap berhasil ketika password, salt, dan iteration count yang tepat digunakan.

Lab 6.2 - Fast hash vs password KDF cost

```
python labs/hash-benchmark.py
```

Interpretation, not benchmark contest

Angka dari laptop lo bukan standard keamanan universal. Hardware, implementation, vectorization, GPU, memory, dan parameter berbeda. Pelajaran utamanya: raw general-purpose hash dirancang cepat; password KDF sengaja membuat tebakan jauh lebih mahal.

7. Online vs Offline Password Attacks

Pattern	Target	Typical constraint
Brute force online	Satu akun / login endpoint	Rate limit, lockout, detection, network latency.
Password spraying	Satu/few password ke banyak akun	Lockout avoidance, detection across identities.
Credential stuffing	Known username/password pairs dari breach lain	Reuse, MFA, bot defenses, anomaly detection.
Offline cracking	Local copy of password verifier/hash	No server rate limit; cost set by KDF + attacker hardware.
Dictionary/rules/masks	Candidate generation strategy	Quality of guesses + keyspace + hash cost.

No online attack lab

Module 15 tidak memberikan workflow spraying/stuffing terhadap endpoint login. Kita hanya mempelajari konsep dan detection telemetry dengan synthetic logs.

Why offline changes the economics

Begitu verifier database bocor, attacker tidak perlu bertanya ke login server untuk setiap guess. Karena itu password storage algorithm dan unique salt sangat penting bahkan bila online rate limiting sudah bagus.

8. Omarchy Tooling: John & Hashcat

```
omarchy pkg add john hashcat

john --list=formats | head
hashcat --version
hashcat -I
```

Di Arch saat ini `john` dan `hashcat` tersedia di official repository Extra. Untuk Hashcat, supported compute backend menentukan apakah GPU/accelerator dapat dipakai; `hashcat -I` adalah first diagnostic. Bila backend belum tersedia, lanjutkan lab dengan John daripada meng-install random runtime dari sumber tidak terpercaya.

Tool	Kelebihan belajar
John the Ripper	Format autodetection, CPU-friendly workflow, wordlist/rules, simple local audit.
Hashcat	Explicit hash modes, GPU/accelerator architecture, mask/rule/keyspace concepts, session/potfile controls.
OpenSSL	Generate deterministic training sha512crypt fixtures.
Python	Policy, KDF, keyspace, telemetry, evidence helpers.

Sensitive artifacts
John potfile, Hashcat potfile/session, wordlists, cracked outputs, and credential CSV harus dianggap sensitive. Jangan commit ke repo publik. Module wrapper menyimpan potfile training di folder lab tersembunyi.

9. Lab Fixture & Workspace

Lab 9.1 - Initialize workspace

```
cd modules/15-passwords-credential-security/labs  
./workspace-init.sh
```

Bundled fixture

```
fixtures/  
├── lab-wordlist.txt  
├── john-sha512crypt.txt  
├── hashcat-sha512crypt.txt  
├── raw-sha256.txt  
├── dummy-credentials.csv  
└── blocklist.txt
```

Fixture dibuat dari password dummy yang memang diketahui dan tidak terhubung ke akun nyata. SHA-512 crypt digunakan untuk menunjukkan salted Unix-style verifier dan didukung John/Hashcat untuk lab.

Lab 9.2 - Regenerate fixture

```
python labs/make-fixtures.py  
sha256sum labs/fixtures/* > module15-credentials/05-evidence/fixtures.sha256
```

Do not replace with /etc/shadow

Jangan gunakan `/etc/shadow` host lo sebagai latihan. Bahkan kalau device milik sendiri, itu mencampur training dengan credential sungguhan dan meningkatkan risiko accidental disclosure.

10. Hash Format & SHA-512 Crypt

Common modular crypt format diawali identifier seperti `\$6\$` untuk SHA-512 crypt. Format menyertakan algorithm identifier, salt, dan derived verifier. Ini berbeda dari raw SHA-512 digest.

```
head -n 1 labs/fixtures/john-sha512crypt.txt
# alice:$6$mod15a$...

python labs/hash-identify.py "$(cut -d: -f2 labs/fixtures/john-sha512crypt.txt | head -n1)"
```

Lab 10.1 - Generate one deterministic training hash

```
openssl passwd -6 -salt module15 'Training123!'
```

Salt yang fixed hanya dipakai agar lab reproducible. Production salt harus unique/random dan dikelola otomatis oleh password-hashing implementation.

Raw fast hash contrast

```
cat labs/fixtures/raw-sha256.txt
```

Algorithm identification needs context

Jangan menentukan impact hanya dari bentuk string. Cari implementation/source/configuration. Aplikasi bisa membungkus format custom, menambahkan pepper, atau memakai migration layer.

11. John the Ripper - Wordlist Audit

John documentation menyarankan wordlist mode sebagai salah satu cracking mode dan menyimpan hasil ke potfile. Lab wrapper sengaja mengunci input ke fixture bundled.

Lab 11.1 - Run safe wrapper

```
./labs/john-lab.sh
```

Lab 11.2 - Display cracked training entries

```
HOME="$PWD/labs/.john-home" john \  
--show --format=sha512crypt \  
labs/fixtures/john-sha512crypt.txt
```

Potfile hygiene

Potfile berisi recovered training secret. Pada engagement real, perlakukan seperti credential evidence: permissions ketat, retention jelas, encrypted storage bila diperlukan, dan jangan masuk Git.

Why wordlists beat blind brute force

Manusia memilih password dengan pola. Wordlist/rules memprioritaskan tebakan plausible. Blind exhaustive search tumbuh eksponensial dan cepat menjadi tidak realistis, terutama terhadap slow/memory-hard KDF.

12. Hashcat - Explicit Modes, Sessions & Potfiles

Hashcat membutuhkan hash mode yang benar. Untuk SHA-512 crypt, mode yang digunakan di lab adalah `1800`. Confirm dengan help/version yang terpasang karena behavior/tooling bisa berubah antar release.

Lab 12.1 - Inspect device backend

```
hashcat -I
```

Lab 12.2 - Bundled hashcat wrapper

```
./labs/hashcat-lab.sh
```

Equivalent core command

```
hashcat -m 1800 -a 0 \  
  labs/fixtures/hashcat-sha512crypt.txt \  
  labs/fixtures/lab-wordlist.txt
```

No GPU obsession

Tujuan modul bukan mengejar benchmark tertinggi. Kalau Hashcat backend tidak usable pada workstation lo, selesaikan learning objectives dengan John dan keyspace tooling. Jangan memasang untrusted driver/runtime hanya demi lab.

13. Rules, Masks & Keyspace Reasoning

Rules memodifikasi candidate word. Masks mendeskripsikan struktur candidate. Keduanya berguna ketika ada evidence tentang pola password; keduanya juga bisa membuang waktu jika assumption salah.

John rule concept

```
john --wordlist=labs/fixtures/lab-wordlist.txt --rules --stdout | head -n 30
```

Hashcat attack-mode concept

```
# dictionary
hashcat -a 0 ...

# mask (training concept)
hashcat -a 3 ... ?l?l?l?d?d
```

Lab 13.1 - Calculate keyspaces first

```
python labs/keyspace.py --charset-size 10 --length 6
python labs/keyspace.py --charset-size 62 --length 10
```

Do the math before the attack

Keyspace = $\text{charset_size}^{\text{length}}$ untuk simple uniform mask. Real guessability berbeda karena user choices tidak uniform, tetapi perhitungan ini membantu menghindari exhaustive plan yang absurd.

14. Password Reuse & Credential Stuffing Risk

Credential reuse mengubah compromise satu service menjadi risk lintas service. Audit reuse secara defensif bisa dilakukan pada authorized dataset tanpa menampilkan plaintext di report.

Lab 14.1 - Dummy reuse audit

```
python labs/credential-reuse.py \
labs/fixtures/dummy-credentials.csv
```

Helper membuat short SHA-256 fingerprint per plaintext hanya untuk grouping dan tidak mencetak password. Fixture sengaja berisi reuse Alice dan Bob.

Hash fingerprint is not anonymization magic

Fingerprint pendek pada low-entropy password bukan perlindungan kuat terhadap attacker yang punya file dan bisa menebak candidates. Di lab helper ini hanya mencegah accidental printing; real credential dataset tetap sensitive.

Controls against reuse-driven compromise

- Unique password per service via password manager.
- MFA / phishing-resistant authenticator where practical.
- Compromised-password blocklist and breach monitoring.
- Anomaly detection for impossible travel / new device / unusual source patterns.

15. MFA, Passkeys & Recovery

MFA menggabungkan independent factors. Password + PIN bukan true MFA karena keduanya knowledge factors. Recovery path harus sekuat atau lebih aman daripada primary authentication.

Factor	Examples	Typical issue
Know	Password, PIN	Phishing, guessing, reuse.
Have	TOTP app, hardware key, certificate	Device/token loss, enrollment/recovery.
Are	Biometric	Privacy, false match, device-bound implementation.
Phishing-resistant public key	FIDO2/WebAuthn/passkey/security key	Account recovery and device lifecycle remain important.

Passwords are not phishing-resistant

NIST explicitly treats passwords as not phishing-resistant. Strong password storage protects verifier breach; it does not stop a user from entering a password into a convincing phishing site.

Recovery threat model

- [] Enrollment requires sufficient assurance.
- [] MFA reset cannot be easier than login.
- [] Recovery codes are secrets with secure storage/rotation.
- [] Lost-device flow produces auditable security events.
- [] High-risk changes trigger reauthentication.

16. Credential Handling & Secure Storage

Red Team dan pentest report sering bersentuhan dengan secrets. Secure-by-design berarti meminimalkan collection, plaintext exposure, duplication, retention, dan uncontrolled sharing.

Artifact	Handling guidance
Recovered password	Avoid recording unless necessary; prefer redacted proof/account count.
Hash dump / verifier	Sensitive evidence; encrypted/controlled storage and strict retention.
Session/API token	Treat as bearer secret; revoke after testing where applicable.
Private key	Never paste into chat/report/repo; protect permissions and revoke/rotate if exposed.
Wordlist built from client data	Sensitive because it may embed names/business context.
Potfile	Sensitive derived credential evidence; exclude from Git.

Repository hygiene

```
cat >> .gitignore <<'EOF'
*.pot
*.potfile
*.rec
**/cracked*
**/credential*
**/loot/**
.env
*.key
*.pem
EOF
```

Environment variables are not universal secret managers

Env vars can be appropriate for short-lived process configuration, but long-lived production secrets should use platform secret stores/secret managers with access control, audit, rotation, and revocation.

17. Blue Team View: Authentication Telemetry

Credential attacks meninggalkan pola di identity logs. Detection tidak hanya menghitung total failure; lihat source, user distribution, success-after-failure, device, ASN/location, MFA event, lockout, token issuance, dan time window.

Lab 17.1 - Synthetic log analysis

```
python labs/auth-log-analyze.py \
  labs/synthetic-auth.log
```

Fixture menghasilkan dua pola: satu source gagal ke beberapa user (spray/stuffing-like telemetry), dan source lain gagal berkali-kali pada satu user (brute-force-like telemetry). Ini synthetic data, bukan attack terhadap service.

Pattern	Possible signal
One source -> many users	Spray/stuffing/bot enumeration candidate.
Many failures -> one user	Targeted guessing or user mistake.
Failure burst -> success	Potential compromised credential; investigate context.
New device + recovery + MFA reset	High-risk account takeover chain.
Token used from unusual source	Session/token abuse candidate.

18. Reporting & Remediation

Credential finding harus menjelaskan root cause dan attacker economics, bukan hanya menulis "password weak".

Finding	Evidence	Remediation direction
Weak password storage	Algorithm/cost/salt implementation evidence	Argon2id/scrypt or appropriate PBKDF2; tune cost; rehash on login.
Common password accepted	Policy test + local blocklist evidence	Compromised/common password screening, longer minimum, password-manager friendly UX.
Credential reuse	Authorized dataset shows same secret across systems	Unique passwords, SSO/federation where appropriate, MFA.
MFA missing on privileged flow	Role/flow evidence	Risk-based / phishing-resistant MFA for sensitive access.
Recovery weaker than login	Reset flow bypasses primary assurance	Harden recovery/enrollment, audit and reauth.
Auth monitoring gap	Synthetic/observed failures not correlated	Centralized identity logs, detection logic, alert tuning.

Lab 18.1 - Finding note

```
python labs/finding-note.py \
  module15-credentials/06-report/reuse.md \
  --title 'Credential reuse across training systems' \
  --evidence 'Two dummy identities reuse credentials in authorized fixture.' \
  --impact 'Compromise of one service may enable access to another.' \
  --remediation 'Use unique generated passwords and MFA.'
```

19. Capstone - Local Credential Security Assessment

Capstone memakai hanya bundle Module 15. Jangan mengganti fixture dengan credential real.

Objective

- [] Initialize workspace dan hash fixture manifest.
- [] Evaluasi tiga example password menggunakan policy-check.
- [] Jelaskan encoding vs raw hash vs sha512crypt dari sample.
- [] Demonstrasikan salt effect dengan PBKDF2 lab.
- [] Crack bundled SHA-512 crypt fixture dengan John; Hashcat optional bila backend tersedia.
- [] Hitung keypace dua candidate strategy dan jelaskan prioritas.
- [] Identifikasi credential reuse pada dummy CSV.
- [] Analisis synthetic auth log dan bedakan spray-like vs brute-force-like pattern.
- [] Buat minimal dua finding dengan remediation.
- [] Generate SHA-256 evidence manifest dan pastikan permissions 600.

Exit criteria

Lo lulus kalau bisa menjelaskan bukan cuma "password berhasil di-crack", tetapi kenapa candidate strategy berhasil, apa peran KDF/salt, apa impact reuse, bagaimana MFA/recovery memengaruhi risk, dan apa remediation production-ready.

Evidence manifest

```
./labs/evidence-manifest.sh \  
  module15-credentials/05-evidence \  
  module15-credentials/05-evidence/sha256-manifest.txt
```

20. Cheat Sheet, Knowledge Check & References

Cheat sheet

Need	Command / concept
Install	omarchy pkg add john hashcat
John fixture	./labs/john-lab.sh
Hashcat fixture	./labs/hashcat-lab.sh
Policy	python labs/policy-check.py "..."
Salt demo	python labs/pbkdf2-demo.py
Keyspace	python labs/keyspace.py --charset-size N --length L
Reuse audit	python labs/credential-reuse.py labs/fixtures/dummy-credentials.csv
Telemetry	python labs/auth-log-analyze.py labs/synthetic-auth.log
Evidence	./labs/evidence-manifest.sh

Knowledge check

1. Kenapa Base64 bukan password protection?
2. Kenapa raw SHA-256 tidak cocok untuk password storage?
3. Apa fungsi salt dan kenapa salt boleh disimpan bersama hash?
4. Apa perbedaan salt dan pepper?
5. Kenapa offline cracking tidak tunduk pada server login rate limit?
6. Apa perbedaan brute force, spraying, dan credential stuffing?
7. Kenapa composition rules modern tidak menjadi fokus utama?
8. Apa perbedaan password + PIN dengan true MFA?
9. Kenapa potfile harus dianggap sensitive?
10. Mengapa cracked password bukan satu-satunya evidence yang penting?

Primary references

- NIST SP 800-63B-4 - Authentication and Authenticator Management: <https://pages.nist.gov/800-63-4/sp800-63b.html>
- OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Credential Stuffing Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Credential_Stuffing_Prevention_Cheat_Sheet.html
- OWASP Multifactor Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html
- John the Ripper documentation: <https://www.openwall.com/john/doc/>
- Hashcat documentation/wiki: <https://hashcat.net/wiki/?id=hashcat>
- Arch Linux Hashcat package: https://archlinux.org/packages/extra/x86_64/hashcat/
- Arch Linux John package/security tracker: <https://security.archlinux.org/package/john>

Next

Module 16 - Linux Privilege Escalation: manual enumeration, sudo, SUID/SGID, capabilities, cron/systemd, writable paths, credentials, containers, and deliberate vulnerable lab workflows.